



中国科学技术大学

程序设计课程回顾与UML简介

徐 伟

E-mail: xuweihf@ustc.edu.cn



程序的开发与编译

❖ Linux下的软件开发

gcc

gcc (GNU Compiler Collection, GNU编译器套件)

GNU开发的编程语言编译器

现已被大多数类Unix操作系统（如Linux、BSD、Mac OS X等）
采纳为标准的编译器，在微软的Windows上也可以使用gcc

gdb

gdb (GNU Project Debugger, GNU项目调试器)

小巧而强大，尺有所短，寸有所长



程序的开发与编译

❖ Linux下的软件开发

🌀 gcc安装

```
sudo apt-get update
```

```
sudo apt-get install gcc
```

🌀 gdb安装

```
sudo apt-get update
```

```
sudo apt-get install gdb
```

示例运行环境 Ubuntu 22.04.1

gcc 7.5.0

gdb 8.1.1



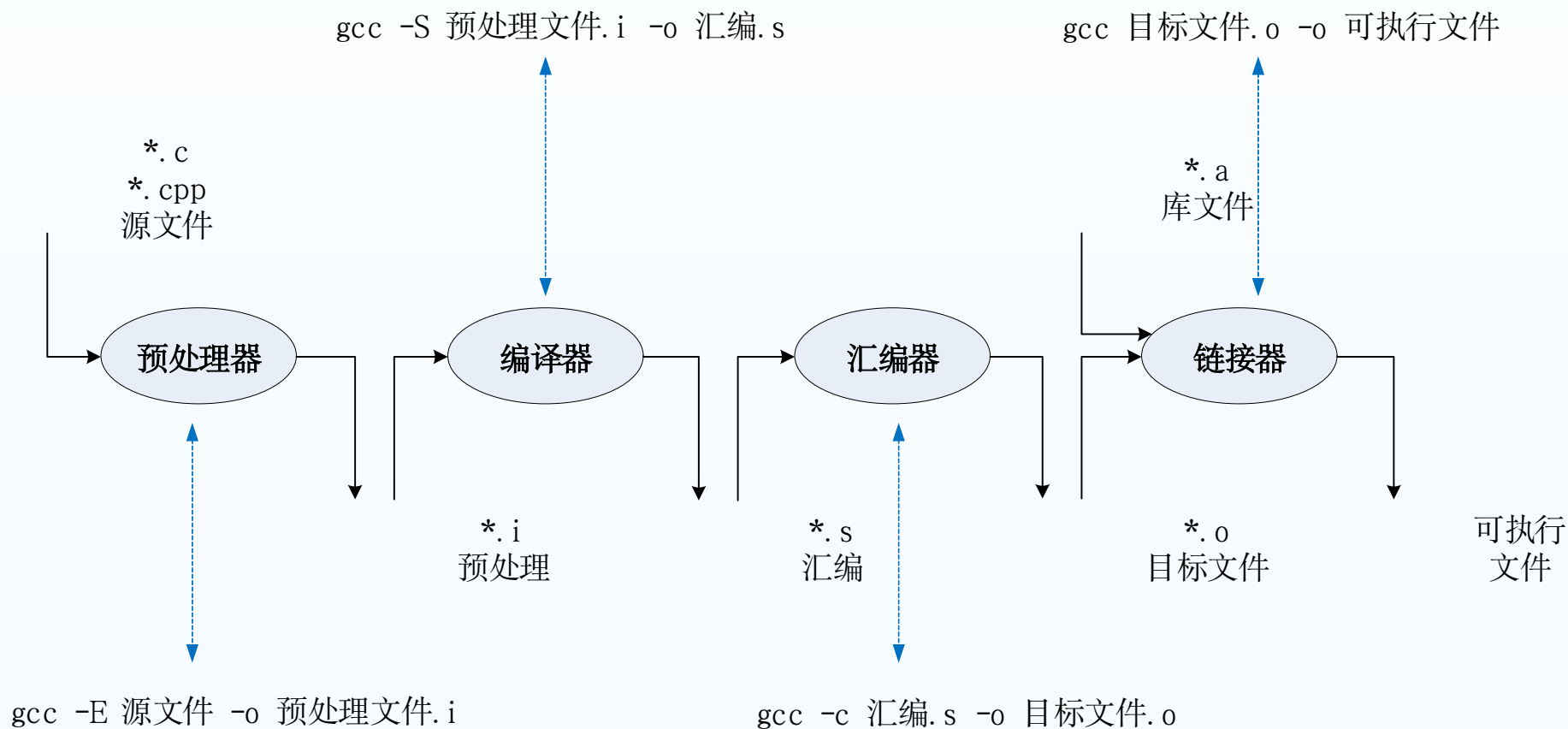
程序的开发与编译

gcc/g++选项	功能	目标文件常用后缀
-E (大写)	预处理指定的源文件，不进行编译。	.i
-S (大写)	编译指定的源文件，生成汇编文件，但是不进行汇编。	.s
-c	编译、汇编指定的源文件，生成目标文件，但是不进行链接。	.o
-o	指定生成目标文件的文件名	
-m32	按照32位进行编译	
-m64	按照64位进行编译	

有关更多编译指令，可自行查看[GCC手册](#)



程序的开发与编译





程序的开发与编译

优化等级	简要说明
-Ofast	在-O3级别的基础上，开启更多激进优化项，该优化等级不会严格遵循语言标准
-O3	在-O2级别的基础上，开启了更多的高级优化项，以编译时间、代码大小、内存为代价获取更高的性能。
-Os	在-O2级别的基础上，开启降低生成代码体量的优化
-O2	开启了大多数中级优化，会改善编译时间开销和最终生成代码性能
-O/-O1	优化效果介于-O0和-O2之间
-O0	默认优化等级，即不开启编译优化，只尝试减少编译时间



gdb入门

❖ gcc文件编译

☞ 需要gdb调试

```
gcc -g MyTestProgram.c
```

默认生成a.out

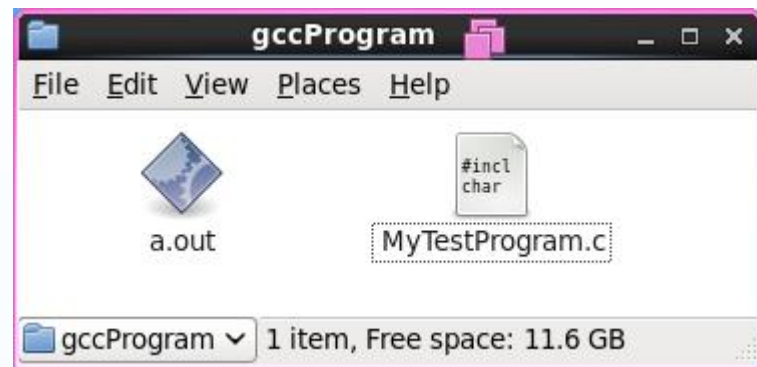
```
gcc -g -o mtp MyTestProgram.c
```

☞ 终端运行 ./mtp ./a.out

☞ 较大工程，推荐使用makefile方式

☞ C++程序采用g++命令进行编译

```
g++ -g MyTestProgram.c
```





gdb

❖ gdb使用

🌀 gdb

file XXX (可执行文件)

■ gdb XXX

(可执行文件)

```
xuwei@localhost:~/Desktop/gccProgram
File Edit View Search Terminal Help
[xuwei@localhost gccProgram]$ gcc -o mtp MyTestProgram.c
[xuwei@localhost gccProgram]$ gcc -g MyTestProgram.c
[xuwei@localhost gccProgram]$ ls
a.out  MyTestProgram.c
[xuwei@localhost gccProgram]$ gdb a.out
GNU gdb (GDB) 7.3.1
Copyright (C) 2011 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-unknown-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /home/xuwei/Desktop/gccProgram/a.out...done.
(gdb) █
```




gdb入门

```
xuwei@localhost:~/Desktop/gccProgram
File Edit View Search Terminal Help
[xuwei@localhost gccProgram]$ gcc -o mtp MyTestProgram.c
[xuwei@localhost gccProgram]$ gcc -g MyTestProgram.c
[xuwei@localhost gccProgram]$ ls
a.out MyTestProgram.c
[xuwei@localhost gccProgram]$ gdb a.out
GNU gdb (GDB) 7.3.1
Copyright (C) 2011 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-unknown-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /home/xuwei/Desktop/gccProgram/a.out...done.
(gdb) █
```



gdb

❖ 断点设置

⌘ break 文件名: 行号

b 文件名: 行号

```
xuwei@localhost:~/Desktop/gccProgram
File Edit View Search Terminal Help
[xuwei@localhost gccProgram]$ gcc -g MyTestProgram.c
[xuwei@localhost gccProgram]$ gdb a.out
GNU gdb (GDB) 7.3.1
Copyright (C) 2011 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-unknown-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /home/xuwei/Desktop/gccProgram/a.out...done.
(gdb) b MyTestProgram.c :15
Breakpoint 1 at 0x40053c: file MyTestProgram.c, line 15.
(gdb) █
```




gdb入门

❖ 断点删除

delete xxx(断点编号)

d xxx(断点编号)

```
xuwei@localhost:~/Desktop/gccProgram
File Edit View Search Terminal Help
This GDB was configured as "x86_64-unknown-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /home/xuwei/Desktop/gccProgram/a.out...done.
(gdb) b MyTestProgram.c :15
Breakpoint 1 at 0x40053c: file MyTestProgram.c, line 15.
(gdb) b MyTestProgram.c :25
Breakpoint 2 at 0x4005bd: file MyTestProgram.c, line 25.
(gdb) i b
Num      Type      Disp Enb Address      What
1        breakpoint keep y   0x000000000040053c in type
                        at MyTestProgram.c:15
2        breakpoint keep y   0x00000000004005bd in main
                        at MyTestProgram.c:25
(gdb) d 2
(gdb) i b
Num      Type      Disp Enb Address      What
1        breakpoint keep y   0x000000000040053c in type
                        at MyTestProgram.c:15
(gdb)
```

The screenshot shows a GDB terminal window. The user has set two breakpoints at lines 15 and 25 of MyTestProgram.c. They then use the 'i b' command to list the breakpoints. Finally, they use the 'd 2' command to delete breakpoint 2, and 'i b' again to confirm that only breakpoint 1 remains. A red box highlights the deletion and verification steps.



gdb入门

❖ 调试运行

run

r

```
xuwei@localhost:~/Desktop/gccProgram
File Edit View Search Terminal Help
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /home/xuwei/Desktop/gccProgram/a.out...done.
(gdb) b MyTestProgram.c :15
Breakpoint 1 at 0x40053c: file MyTestProgram.c, line 15.
(gdb) b MyTestProgram.c :25
Breakpoint 2 at 0x4005bd: file MyTestProgram.c, line 25.
(gdb) i b
Num      Type           Disp Enb Address                What
1        breakpoint     keep y   0x000000000040053c    in type
                                     at MyTestProgram.c:15
2        breakpoint     keep y   0x00000000004005bd    in main
                                     at MyTestProgram.c:25
(gdb) d 2
(gdb) i b
Num      Type           Disp Enb Address                What
1        breakpoint     keep y   0x000000000040053c    in type
                                     at MyTestProgram.c:15
(gdb) r
Starting program: /home/xuwei/Desktop/gccProgram/a.out
```




gdb入门

```
xuwei@localhost:~/Desktop/gccProgram
File Edit View Search Terminal Help
Breakpoint 2 at 0x4005bd: file MyTestProgram.c, line 25.
(gdb) i b
Num      Type      Disp Enb Address      What
1        breakpoint keep y   0x000000000040053c in type
                                at MyTestProgram.c:15
2        breakpoint keep y   0x00000000004005bd in main
                                at MyTestProgram.c:25
(gdb) d 2
(gdb) i b
Num      Type      Disp Enb Address      What
1        breakpoint keep y   0x000000000040053c in type
                                at MyTestProgram.c:15
(gdb) r
Starting program: /home/xuwei/Desktop/gccProgram/a.out
1750

Breakpoint 1, type (m=2000) at MyTestProgram.c:15
15          if(m % 4 != 0 || (m % 100 == 0 && m % 400 != 0))
return 0; //00000000
(gdb) █
```



gdb入门

❖ 参数值查看

❧ print xxx(变量名)

p xxx(变量名)

```
xuwei@localhost:~/Desktop/gccProgram
File Edit View Search Terminal Help

2      breakpoint      keep y   0x000000000004005bd in main
                                           at MyTestProgram.c:15
                                           at MyTestProgram.c:25

(gdb) d 2
(gdb) i b
Num      Type      Disp Enb Address      What
1        breakpoint keep y   0x0000000000040053c in type
                                           at MyTestProgram.c:15

(gdb) r
Starting program: /home/xuwei/Desktop/gccProgram/a.out
1750

Breakpoint 1, type (m=2000) at MyTestProgram.c:15
15      if(m % 4 != 0 || (m % 100 == 0 && m % 400 != 0))
return 0; //00000000

(gdb) print m
$1 = 2000
(gdb) p m
$2 = 2000
(gdb) █
```



gdb入门

❖ 修改参数值

☞ `print xxx(变量名) = XXX`

☞ `p xxx(变量名) = XXX`

```
-U: **- *gud-a.out* Bot (40,6) (Debugger:run [stopped])
38 (gdb) p array
39 $2 = {1, 2, 3}
40 (gdb) p array[1] = 12
41 $3 = 12
42 (gdb) p array
43 $4 = {1, 12, 3}
44 (gdb)
```




gdb入门

```
xuwei@localhost:~/Desktop/gccProgram
File Edit View Search Terminal Help

2          breakpoint      keep y   0x0000000000004005bd in main
                                           at MyTestProgram.c:15
                                           at MyTestProgram.c:25
(gdb) d 2
(gdb) i b
Num      Type          Disp Enb Address          What
1        breakpoint    keep y   0x00000000000040053c in type
                                           at MyTestProgram.c:15
(gdb) r
Starting program: /home/xuwei/Desktop/gccProgram/a.out
1750

Breakpoint 1, type (m=2000) at MyTestProgram.c:15
15          if(m % 4 != 0 || (m % 100 == 0 && m % 400 != 0))
return 0; //00000000
(gdb) print m
$1 = 2000
(gdb) p m
$2 = 2000
(gdb) █
```



gdb入门

❖ 函数调用堆栈

∞ bt

```
xuwei@localhost:~/Desktop/gccProgram
File Edit View Search Terminal Help
(gdb) d 2
(gdb) i b
Num      Type      Disp Enb Address      What
1        breakpoint keep y  0x000000000040053c in type
                                     at MyTestProgram.c:15
(gdb) r
Starting program: /home/xuwei/Desktop/gccProgram/a.out
1750

Breakpoint 1, type (m=2000) at MyTestProgram.c:15
15      if(m % 4 != 0 || (m % 100 == 0 && m % 400 != 0))
return 0; //00000000
(gdb) print m
$1 = 2000
(gdb) p m
$2 = 2000
(gdb) bt
#0  type (m=2000) at MyTestProgram.c:15
#1  0x000000000040062d in main () at MyTestProgram.c:27
(gdb) █
```



gdb入门

❖ 单步执行

❧ next n

❧ step s

❖ 继续执行到下一个断点

❧ continue c

❖ 查看文件代码

❧ list l

❖ 退出gdb

❧ quit q



体系结构黄金时代的机会

❖ 软件为中心

- ❧ 现代编程语言很多是script语言-> 需要解析、动态类型、代码复用
- ❧ 编程高效但是执行不见得高效: Python vs. C

❖ 硬件为中心

- ❧ 唯一可行的路径是领域专用架构
- ❧ 一颗芯片只做一部分事情, 但是性能很好

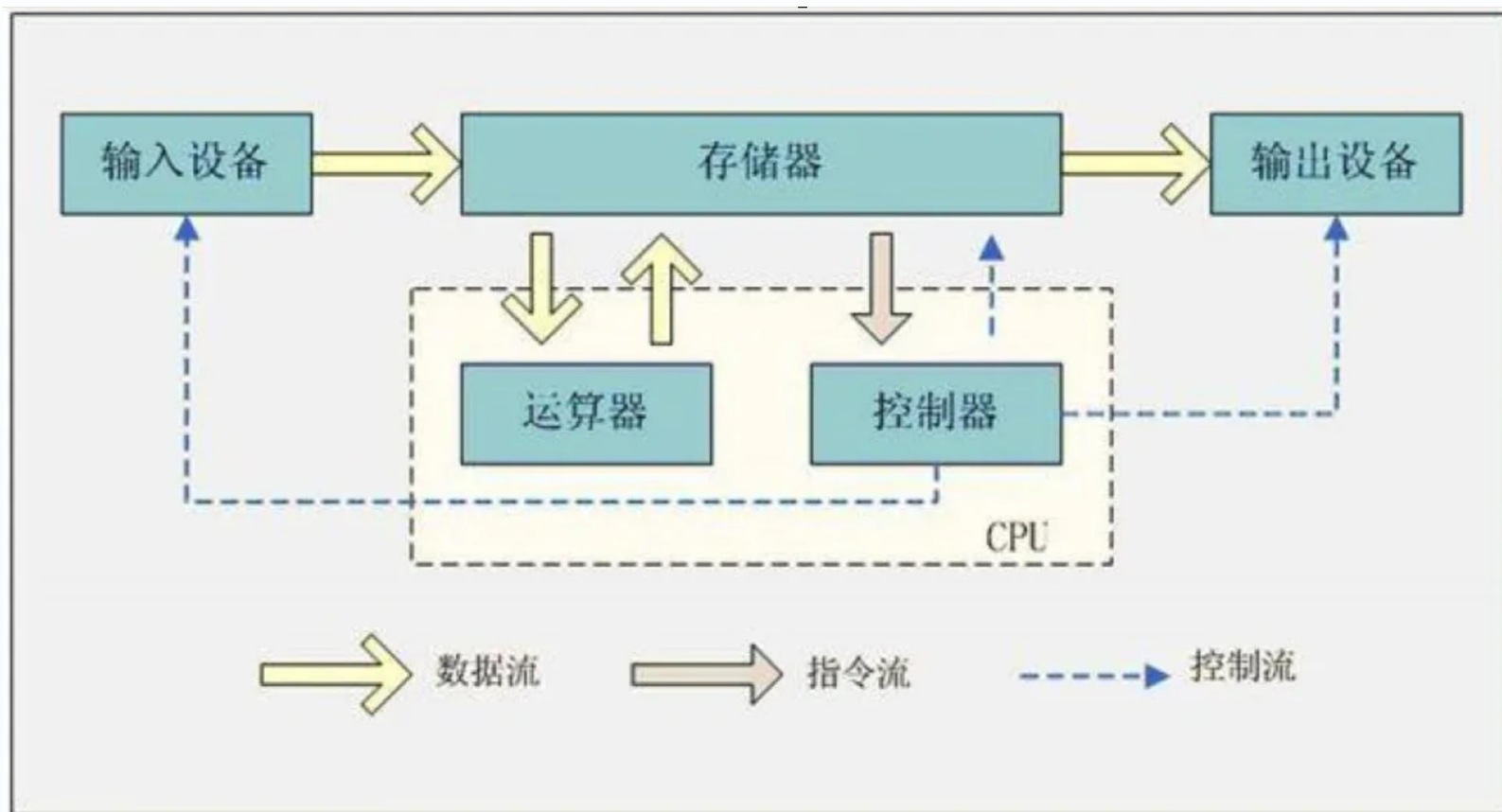
❖ 软硬协同的设计思路

- ❧ 专用领域语言+专用领域架构: CUDA + GPU
- ❧ 可以最大化挖掘硬件的并行度



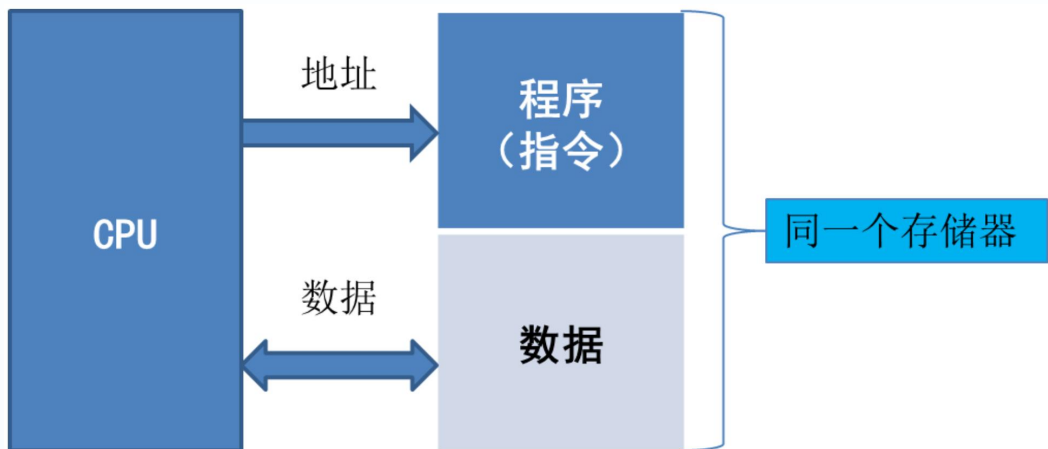
程序是如何运行起来的

❖ 计算机

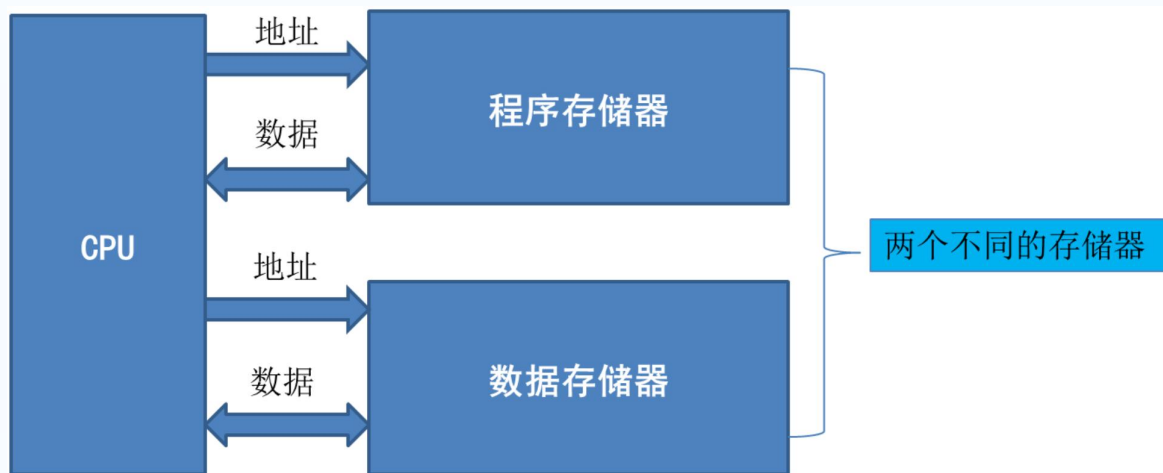




程序是如何运行起来的



冯诺依曼体系结构

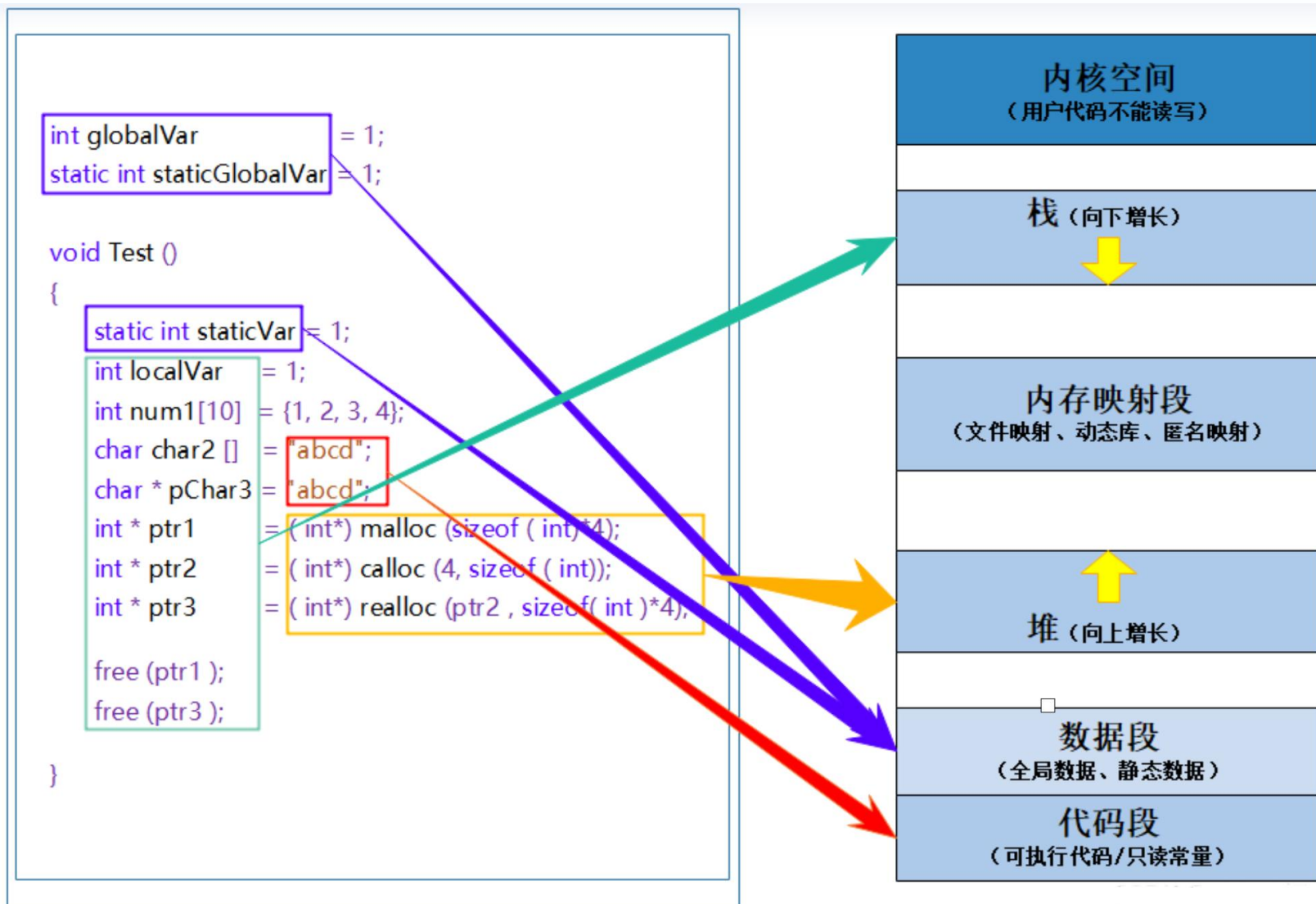


哈佛体系结构



程序是如何运行起来的

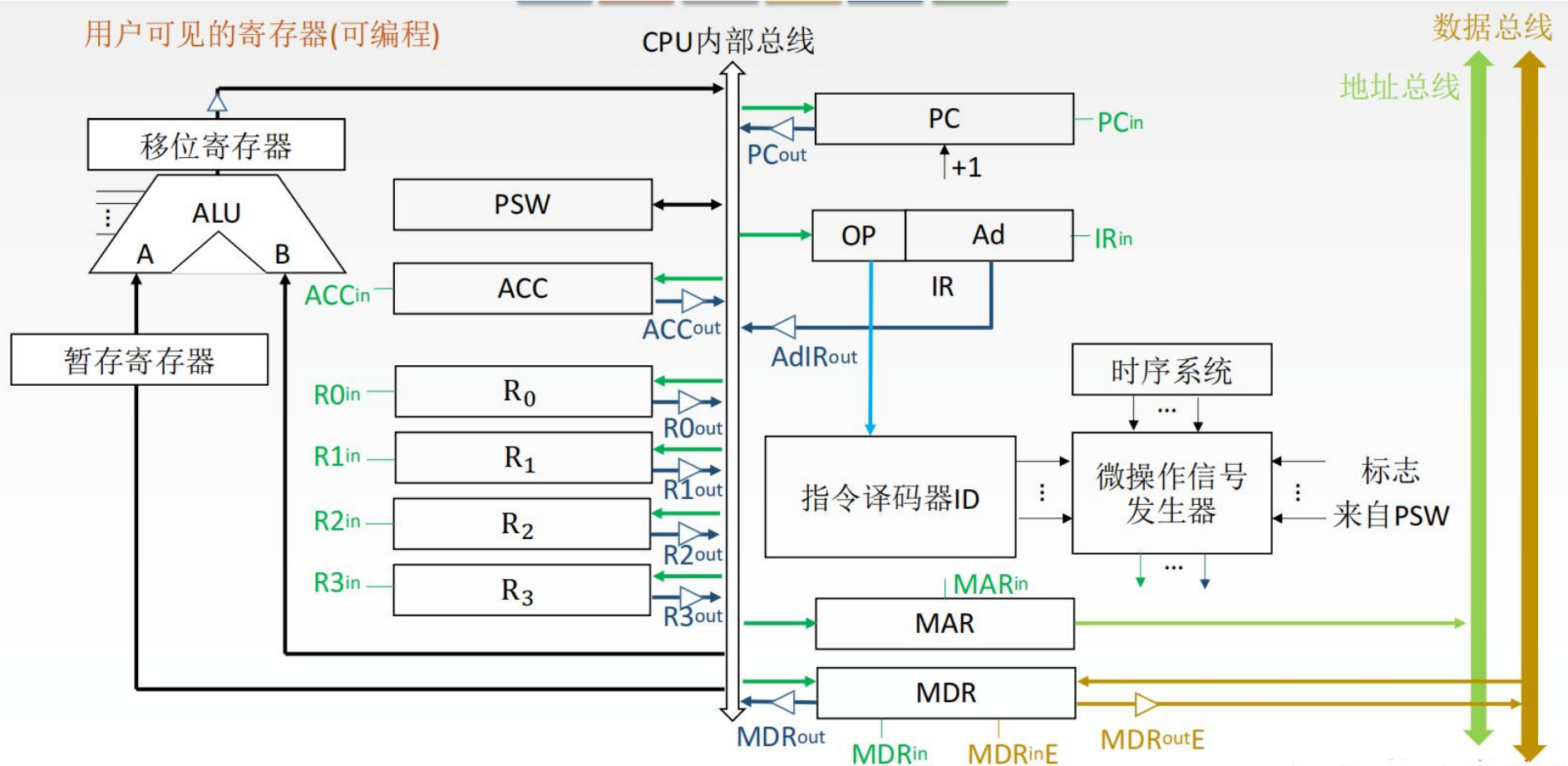
❖ 内存:





程序是如何运行起来的

❖ CPU结构

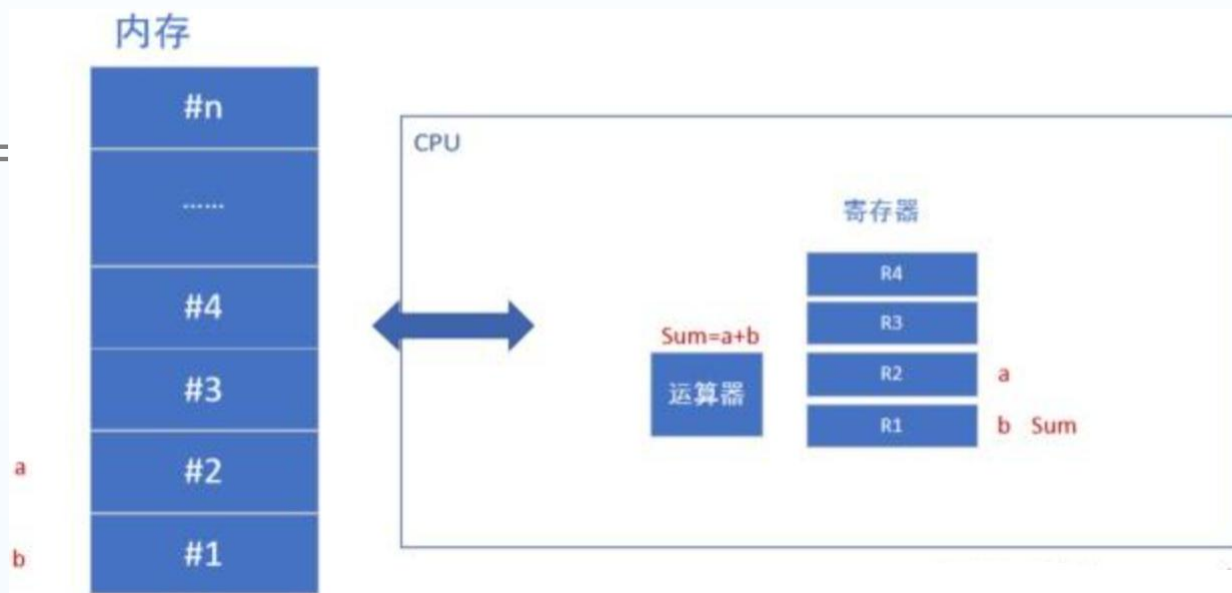




程序是如何运行起来的

❖ CPU和内存的交互

```
#include <stdio.h>
int main ()
{
    int a = 5, b =
    sum = a + b;
    printf("%d\n",
    return 0;
}
```





流程图

- ❖ 起源：工业工程与流程图表（1920s）
 - ❧ 最初是为了分析建筑工人的动作和工序，试图找到最有效的施工方法，减少不必要的动作
 - ❧ 这被认为是流程图的雏形
- ❖ 标准化：ASME与符号规范的建立（1940s）
 - ❧ 美国机械工程师学会在20世纪40年代开始推动流程图的标准
 - ❧ 流程图已成为工业生产、商业管理和后勤领域中描述工作流的通用工具



流程图

- ❖ 引入计算机领域：冯·诺依曼与程序框图（1940s-1950s）
 - ❧ 流程图因其直观、清晰的特性，被引入计算机编程领域
 - ❧ 流程图是程序员最主要的设计和交流工具
- ❖ 黄金时代与标准化：ISO与ANSI（1960s-1970s）
 - ❧ 1970年，美国国家标准协会采用了基于之前工作的标准，制定了 **ANSI X3.5** 标准《信息系统的流程图符号》
 - ❧ 1970年，ISO也采纳了ANSI标准，使得流程图符号在全球范围内得到统一



流程图

- ❖ 现代演变：从流程图到UML活动图（1990s至今）
 - ❧ 1997年，统一建模语言发布。它将传统流程图的思想吸收并扩展，形成了 UML活动图（支持并发等特性）

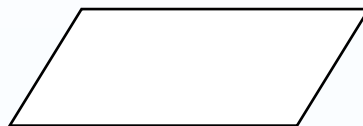


流程图

❖ 基本符号



开始/
结束



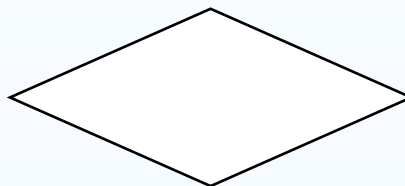
数据输入
输出



处理



准备或
预处理



条件判断



文件或
文档

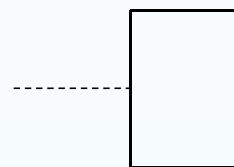


流程图

❖ 基本符号



流线

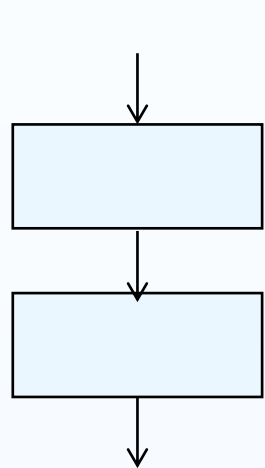


注解或注
释

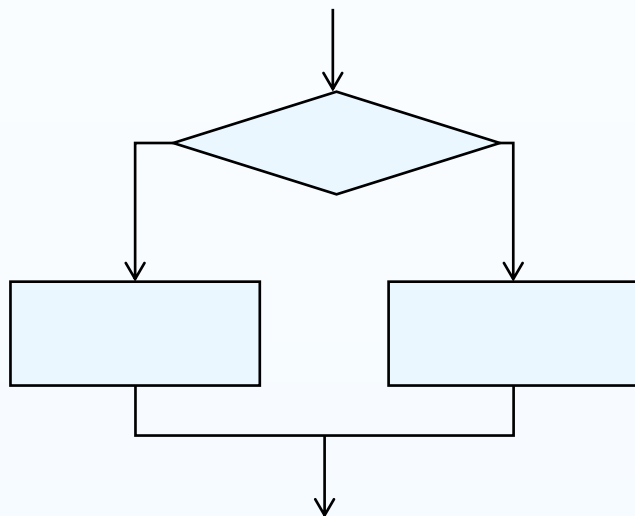


流程图

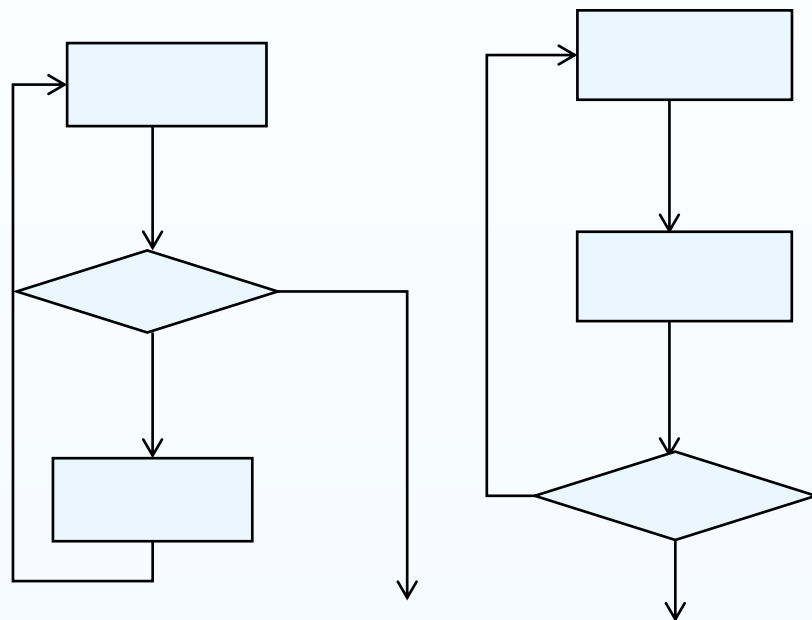
❖ 基本结构



顺序



分支



循环



流程图

❖ 注意：

- ❧ 请使用标准的框图符号，而不是自定义符号
- ❧ 至少包括开始、结束和一个处理
- ❧ 整体绘制方向是从上到下、从左往右
- ❧ 除了判断框外，大多数框图只有一个流入和一个流出
- ❧ 开始框只有一个流出线，终止框只有一个流入线
- ❧ 判断框输出线需要标注判断条件
- ❧ 线与线避免交叉
- ❧ 语言尽可能精炼



流程图

❖ 示例

